

Pandas Complete Reference Manual

Every topic and subtopic explained in two lines, with four runnable examples and real outputs each.

By PRAVEEN VIJAYAPURAM

1. Introduction to Pandas

1.1 Importing Pandas

Pandas is imported using the standard alias `pd`. This alias is used throughout the entire pandas ecosystem and documentation.

Question 1: Write a program to import pandas and print the installed version.

Input:

```
import pandas as pd
print("Pandas successfully loaded version:", pd.__version__)
```

Output:

```
Pandas successfully loaded version: 3.0.2
```

Question 2: Write a program to check whether pandas has been imported successfully.

Input:

```
import pandas as pd
print("Is pandas imported?", 'pd' in dir())
```

Output:

```
Is pandas imported? True
```

Question 3: Write a program to print the name of the pandas module.

Input:

```
import pandas as pd
print("Pandas module name:", pd.__name__)
```

Output:

```
Pandas module name: pandas
```

Question 4: Write a program to print the file path of the installed pandas module.

Input:

```
import pandas as pd
print("Pandas file location:", pd.__file__[-30:])
```

Output:

```
Pandas file location: st-packages/pandas/__init__.py
```

1.2 Creating a Series

A pandas Series is a one-dimensional labeled array capable of holding any data type. Each value is automatically paired with an index label.

Question 1: Write a program to create a Series from a list of integers.

Input:

```
import pandas as pd
my_list = [5, 10, 15, 20]
my_series = pd.Series(my_list)
print("Created Series:\n", my_series)
```

Output:

```
Created Series:
0 5
1 10
2 15
3 20
dtype: int64
```

Question 2: Write a program to create a Series from a list of strings.

Input:

```
import pandas as pd
names_series = pd.Series(["a", "b", "c"])
print("String Series:\n", names_series)
```

Output:

```
String Series:
0 a
1 b
2 c
dtype: str
```

Question 3: Write a program to create a Series with a custom index.

Input:

```
import pandas as pd
custom_index = pd.Series([100, 200, 300], index=["x", "y", "z"])
print("Custom Index Series:\n", custom_index)
```

Output:

```
Custom Index Series:
x 100
y 200
z 300
dtype: int64
```

Question 4: Write a program to create a Series from a list of floating-point numbers.

Input:

```
import pandas as pd
float_series = pd.Series([1.1, 2.2, 3.3])
print("Float Series:\n", float_series)
```

Output:

```
Float Series:
0 1.1
1 2.2
2 3.3
dtype: float64
```

1.3 Creating a DataFrame

A pandas DataFrame is a two-dimensional labeled data structure with columns that can each hold a different data type, similar to a spreadsheet.

Question 1: Write a program to create a DataFrame from a dictionary of names and ages.

Input:

```
import pandas as pd
data = {"Name": ["Ann", "Ben", "Cid"], "Age": [23, 31, 27]}
df = pd.DataFrame(data)
```

```
print("Created DataFrame:\n", df)
```

Output:

```
Created DataFrame:  
Name Age  
0 Ann 23  
1 Ben 31  
2 Cid 27
```

Question 2: Write a program to create a DataFrame from a nested list.

Input:

```
import pandas as pd  
nested_list = [[1, 2], [3, 4]]  
matrix_df = pd.DataFrame(nested_list, columns=["A", "B"])  
print("2D DataFrame:\n", matrix_df)
```

Output:

```
2D DataFrame:  
A B  
0 1 2  
1 3 4
```

Question 3: Write a program to create a DataFrame with a single float column.

Input:

```
import pandas as pd  
df = pd.DataFrame({"Score": [88.5, 92.1, 76.4]})  
print("Float DataFrame:\n", df)
```

Output:

```
Float DataFrame:  
Score  
0 88.5  
1 92.1  
2 76.4
```

Question 4: Write a program to create a DataFrame with a custom row index.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]}, index=["r1", "r2"])
print("Custom Row Index:\n", df)
```

Output:

```
Custom Row Index:
A B
r1 1 3
r2 2 4
```

2. DataFrame Creation Functions

2.1 pd.DataFrame() from dict of lists

A dictionary of equal-length lists can be converted directly into a DataFrame, where each key becomes a column name.

Question 1: Write a program to create a DataFrame of city names and temperatures from a dictionary of lists.

Input:

```
import pandas as pd
data = {"City": ["Delhi", "Pune", "Goa"], "Temp": [30, 27, 32]}
df = pd.DataFrame(data)
print(df)
```

Output:

```
City Temp
0 Delhi 30
1 Pune 27
2 Goa 32
```

Question 2: Write a program to list the column names of a DataFrame created from a dictionary.

Input:

```
import pandas as pd
data = {"Item": ["Pen", "Book"], "Price": [10, 250]}
df = pd.DataFrame(data)
print(df.columns.tolist())
```

Output:

```
['Item', 'Price']
```

Question 3: Write a program to print the shape of a single-column DataFrame.

Input:

```
import pandas as pd
data = {"A": [1, 2, 3]}
df = pd.DataFrame(data)
print(df.shape)
```

Output:

```
(3, 1)
```

Question 4: Write a program to create a DataFrame with three numeric columns.

Input:

```
import pandas as pd
data = {"X": [1, 2], "Y": [3, 4], "Z": [5, 6]}
df = pd.DataFrame(data)
print(df)
```

Output:

```
X Y Z
0 1 3 5
1 2 4 6
```

2.2 pd.DataFrame() from list of dicts

A list where each element is a dictionary can also build a DataFrame, with each dictionary becoming one row. Missing keys become NaN values.

Question 1: Write a program to create a DataFrame from a list of dictionaries with matching keys.

Input:

```
import pandas as pd
records = [{"a": 1, "b": 2}, {"a": 3, "b": 4}]
df = pd.DataFrame(records)
print(df)
```

Output:

```
a b
0 1 2
1 3 4
```

Question 2: Write a program to create a DataFrame of names and ages from a list of dictionaries.

Input:

```
import pandas as pd
records = [{"name": "Raj", "age": 21}, {"name": "Sam", "age": 25}]
df = pd.DataFrame(records)
print(df)
```

Output:

```
name age
0 Raj 21
1 Sam 25
```

Question 3: Write a program to create a DataFrame from a list of dictionaries with missing keys.

Input:

```
import pandas as pd
records = [{"a": 1}, {"a": 2, "b": 5}]
df = pd.DataFrame(records)
print(df)
```

Output:

```
a b
0 1 NaN
1 2 5.0
```

Question 4: Write a program to print the data types of a DataFrame created from a list of dictionaries.

Input:

```
import pandas as pd
records = [{"x": 1, "y": 2}]
df = pd.DataFrame(records)
print(df.dtypes)
```

Output:

```
x int64
y int64
dtype: object
```

2.3 pd.date_range()

pd.date_range() generates a sequence of evenly spaced dates or timestamps between a start and end point. It is often used to build a DateTimeIndex for time series data.

Question 1: Write a program to generate five consecutive daily dates using pd.date_range().

Input:

```
import pandas as pd
dates = pd.date_range(start="2024-01-01", periods=5)
print(dates)
```

Output:

```
DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04',
               '2024-01-05'], dtype='datetime64[us]', freq='D')
```

Question 2: Write a program to generate dates spaced two days apart between two given dates.

Input:

```
import pandas as pd
dates = pd.date_range(start="2024-01-01", end="2024-01-10", freq="2D")
print(dates)
```

Output:

```
DatetimeIndex(['2024-01-01', '2024-01-03', '2024-01-05', '2024-01-07',  
'2024-01-09'], dtype='datetime64[us]', freq='2D')
```

Question 3: Write a program to generate four month-end dates using `pd.date_range()`.

Input:

```
import pandas as pd  
months = pd.date_range(start="2024-01-01", periods=4, freq="ME")  
print(months)
```

Output:

```
DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-31', '2024-04-30'],  
dtype='datetime64[us]', freq='ME')
```

Question 4: Write a program to generate three consecutive hourly timestamps.

Input:

```
import pandas as pd  
hours = pd.date_range(start="2024-01-01", periods=3, freq="h")  
print(hours)
```

Output:

```
DatetimeIndex(['2024-01-01 00:00:00', '2024-01-01 01:00:00',  
'2024-01-01 02:00:00'], dtype='datetime64[us]', freq='h')
```

2.4 `pd.read_csv()` & `pd.to_csv()` (in-memory example)

`pd.read_csv()` loads tabular data from a CSV file into a `DataFrame`, and `DataFrame.to_csv()` writes a `DataFrame` back out to a CSV file on disk.

Question 1: Write a program to write a `DataFrame` to a CSV file and read it back.

Input:

```
import pandas as pd  
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})  
df.to_csv("sample.csv", index=False)  
loaded = pd.read_csv("sample.csv")
```

```
print(loaded)
```

Output:

```
A B
0 1 3
1 2 4
```

Question 2: Write a program to save a DataFrame of names and ages to CSV and print its shape after reloading.

Input:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Ann", "Ben"], "Age": [23, 31]})
df.to_csv("people.csv", index=False)
loaded = pd.read_csv("people.csv")
print(loaded.shape)
```

Output:

```
(2, 2)
```

Question 3: Write a program to write a float column to CSV and check its data type after reloading.

Input:

```
import pandas as pd
df = pd.DataFrame({"Val": [1.5, 2.5, 3.5]})
df.to_csv("vals.csv", index=False)
loaded = pd.read_csv("vals.csv")
print(loaded.dtypes)
```

Output:

```
Val float64
dtype: object
```

Question 4: Write a program to write a CSV file without headers and read it back with custom column names.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [10, 20, 30]})
df.to_csv("nohdr.csv", index=False, header=False)
loaded = pd.read_csv("nohdr.csv", header=None, names=["A"])
print(loaded)
```

Output:

```
A
0 10
1 20
2 30
```

3. Series and DataFrame Attributes

3.1 shape

The `.shape` attribute reports the number of rows and columns of a DataFrame as a tuple, or the number of elements of a Series.

Question 1: Write a program to print the shape of a two-column DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
print("DataFrame Shape:", df.shape)
```

Output:

```
DataFrame Shape: (3, 2)
```

Question 2: Write a program to print the shape of a Series.

Input:

```
import pandas as pd
s_obj = pd.Series([1, 2, 3, 4])
print(s_obj.shape)
```

Output:

```
(4,)
```

Question 3: Write a program to print the shape of a three-column, single-row DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1], "B": [2], "C": [3]})
print(df.shape)
```

Output:

```
(1, 3)
```

Question 4: Write a program to print the shape of an empty DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame()
print(df.shape)
```

Output:

```
(0, 0)
```

3.2 columns & index

The `.columns` attribute lists the labels of every column, while the `.index` attribute lists the labels of every row.

Question 1: Write a program to print the column labels of a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Ann"], "Age": [23]})
print("Columns:", df.columns.tolist())
```

Output:

```
Columns: ['Name', 'Age']
```

Question 2: Write a program to print the index labels of a DataFrame with a custom index.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]}, index=["x", "y", "z"])
print(df.index.tolist())
```

Output:

```
['x', 'y', 'z']
```

Question 3: Write a program to print the default index of a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(df.index)
```

Output:

```
RangeIndex(start=0, stop=2, step=1)
```

Question 4: Write a program to rename the columns of a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1]})
df.columns = ["RenamedA"]
print(df.columns.tolist())
```

Output:

```
['RenamedA']
```

3.3 dtypes

The `.dtypes` attribute reveals the data type stored in each column of a DataFrame, such as `int64`, `float64`, or `object`.

Question 1: Write a program to print the data types of every column in a mixed-type DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [1.5, 2.5], "C": ["x", "y"]})
print(df.dtypes)
```

Output:

```
A int64
B float64
C str
dtype: object
```

Question 2: Write a program to print the data type of a boolean column.

Input:

```
import pandas as pd
df = pd.DataFrame({"Flag": [True, False]})
print(df.dtypes)
```

Output:

```
Flag bool
dtype: object
```

Question 3: Write a program to print the data type of a Series.

Input:

```
import pandas as pd
s_obj = pd.Series([1, 2, 3])
print(s_obj.dtype)
```

Output:

```
int64
```

Question 4: Write a program to convert a column to float and print its data type.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
df["A"] = df["A"].astype(float)
print(df.dtypes)
```

Output:

```
A float64  
dtype: object
```

3.4 info() & describe()

info() prints a concise summary of a DataFrame's structure, and describe() computes summary statistics for its numeric columns.

Question 1: Write a program to print a summary of a DataFrame using info().

Input:

```
import pandas as pd  
df = pd.DataFrame({"A": [1, 2, 3], "B": [4.0, 5.0, 6.0]})  
df.info()
```

Output:

```
<class 'pandas.DataFrame'>  
RangeIndex: 3 entries, 0 to 2  
Data columns (total 2 columns):  
# Column Non-Null Count Dtype  
-----  
0 A 3 non-null int64  
1 B 3 non-null float64  
dtypes: float64(1), int64(1)  
memory usage: 180.0 bytes
```

Question 2: Write a program to print summary statistics of a numeric column using describe().

Input:

```
import pandas as pd  
df = pd.DataFrame({"Score": [70, 85, 90, 60]})  
print(df.describe())
```

Output:

```
Score  
count 4.000000  
mean 76.250000  
std 13.768926  
min 60.000000  
25% 67.500000
```

```
50% 77.500000
75% 86.250000
max 90.000000
```

Question 3: Write a program to print only the mean and standard deviation from describe().

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3, 4, 5]})
print(df.describe().loc[["mean", "std"]])
```

Output:

```
A
mean 3.000000
std 1.581139
```

Question 4: Write a program to print info() for a DataFrame containing a missing value.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, None, 3]})
df.info()
```

Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 1 columns):
# Column Non-Null Count Dtype
---
0 A 2 non-null float64
dtypes: float64(1)
memory usage: 156.0 bytes
```

4. Indexing and Selection

4.1 Column Selection

A single column can be selected from a DataFrame using square-bracket notation, which returns that column as a Series.

Question 1: Write a program to select a single column from a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Ann", "Ben"], "Age": [23, 31]})
print("Name Column:\n", df["Name"])
```

Output:

```
Name Column:
0 Ann
1 Ben
Name: Name, dtype: str
```

Question 2: Write a program to select multiple columns from a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(df[["A", "B"]])
```

Output:

```
A B
0 1 3
1 2 4
```

Question 3: Write a program to check the type of a selected column.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(type(df["A"]))
```

Output:

```
<class 'pandas.Series'>
```

Question 4: Write a program to create a new column from an existing one.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
df["B"] = df["A"] * 2
print(df)
```

Output:

```
A B
0 1 2
1 2 4
2 3 6
```

4.2 loc[] (Label-based)

The .loc[] indexer selects rows and columns using their explicit labels rather than integer positions.

Question 1: Write a program to select a row by its label using loc[].

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [10, 20, 30]}, index=["r1", "r2", "r3"])
print("Row r2:\n", df.loc["r2"])
```

Output:

```
Row r2:
A 20
Name: r2, dtype: int64
```

Question 2: Write a program to select a specific cell using loc[].

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
print(df.loc[0, "B"])
```

Output:

```
4
```

Question 3: Write a program to select a range of rows and columns using `loc[]`.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
print(df.loc[0:1, ["A", "B"]])
```

Output:

```
A B
0 1 4
1 2 5
```

Question 4: Write a program to update a cell value using `loc[]`.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
df.loc[1, "A"] = 99
print(df)
```

Output:

```
A
0 1
1 99
2 3
```

4.3 `iloc[]` (Position-based)

The `.iloc[]` indexer selects rows and columns purely by integer position, similar to standard Python list slicing.

Question 1: Write a program to select a row by position using `iloc[]`.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [10, 20, 30], "B": [40, 50, 60]})
print("Row at position 1:\n", df.iloc[1])
```

Output:

```
Row at position 1:  
A 20  
B 50  
Name: 1, dtype: int64
```

Question 2: Write a program to select a specific cell by position using `iloc[]`.

Input:

```
import pandas as pd  
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})  
print(df.iloc[0, 1])
```

Output:

```
4
```

Question 3: Write a program to select the first two rows using `iloc[]`.

Input:

```
import pandas as pd  
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})  
print(df.iloc[:2, :])
```

Output:

```
A B  
0 1 4  
1 2 5
```

Question 4: Write a program to select the last row using `iloc[]`.

Input:

```
import pandas as pd  
df = pd.DataFrame({"A": [1, 2, 3]})  
print(df.iloc[-1])
```

Output:

```
A 3  
Name: 2, dtype: int64
```

4.4 Boolean Indexing (Filtering)

Boolean indexing uses a True/False condition to keep only the rows that satisfy it, with no explicit loop needed.

Question 1: Write a program to filter rows where a score is above 50.

Input:

```
import pandas as pd
df = pd.DataFrame({"Score": [45, 89, 23, 71]})
print("Scores above 50:\n", df[df["Score"] > 50])
```

Output:

```
Scores above 50:
Score
1  89
3  71
```

Question 2: Write a program to filter rows with even values.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3, 4, 5, 6]})
print(df[df["A"] % 2 == 0])
```

Output:

```
A
1  2
3  4
5  6
```

Question 3: Write a program to filter out rows matching a specific city.

Input:

```
import pandas as pd
df = pd.DataFrame({"City": ["Delhi", "Pune", "Goa"], "Temp": [30, 27, 35]})
print(df[df["City"] != "Pune"])
```

Output:

```
City Temp
0 Delhi 30
2 Goa 35
```

Question 4: Write a program to filter rows with positive values.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [-5, 3, -2, 8]})
print(df[df["A"] > 0])
```

Output:

```
A
1 3
3 8
```

5. Data Types & Conversion

5.1 astype()

The astype() method returns a new Series or DataFrame with every value converted to a specified data type.

Question 1: Write a program to convert a float Series to integers.

Input:

```
import pandas as pd
s_obj = pd.Series([10.9, 20.1, 30.5])
print("Converted to int:\n", s_obj.astype(int))
```

Output:

```
Converted to int:
0 10
1 20
2 30
dtype: int64
```

Question 2: Write a program to convert a column to float using astype().

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
print(df["A"].astype(float))
```

Output:

```
0 1.0
1 2.0
2 3.0
Name: A, dtype: float64
```

Question 3: Write a program to convert a Series to boolean values.

Input:

```
import pandas as pd
s_obj = pd.Series([0, 1, 2])
print(s_obj.astype(bool))
```

Output:

```
0 False
1 True
2 True
dtype: bool
```

Question 4: Write a program to convert a Series to strings.

Input:

```
import pandas as pd
s_obj = pd.Series([1, 2, 3])
print(s_obj.astype(str))
```

Output:

```
0 1
1 2
2 3
dtype: str
```

5.2 pd.to_numeric() & pd.to_datetime()

pd.to_numeric() converts values into numbers, while pd.to_datetime() converts values into proper Timestamp objects.

Question 1: Write a program to convert a Series of numeric strings to numbers.

Input:

```
import pandas as pd
s_obj = pd.Series(["1", "2", "3"])
print(pd.to_numeric(s_obj))
```

Output:

```
0 1
1 2
2 3
dtype: int64
```

Question 2: Write a program to convert a Series of date strings to datetime objects.

Input:

```
import pandas as pd
s_obj = pd.Series(["2024-01-01", "2024-02-15"])
print(pd.to_datetime(s_obj))
```

Output:

```
0 2024-01-01
1 2024-02-15
dtype: datetime64[us]
```

Question 3: Write a program to convert a Series of mixed numeric strings to numbers.

Input:

```
import pandas as pd
s_obj = pd.Series(["10", "20.5", "30"])
print(pd.to_numeric(s_obj))
```

Output:

```
0 10.0
1 20.5
2 30.0
dtype: float64
```

Question 4: Write a program to print the day name of a converted date.

Input:

```
import pandas as pd
dt = pd.to_datetime("2024-07-04")
print(dt.day_name())
```

Output:

Thursday

6. Basic Arithmetic & Operations

6.1 Element-wise Operations

Arithmetic operators like +, -, *, and / apply automatically to every matching pair of values across two same-shaped Series or DataFrames.

Question 1: Write a program to add two Series element-wise.

Input:

```
import pandas as pd
a = pd.Series([1, 2, 3])
b = pd.Series([10, 20, 30])
print("Element-wise Addition:\n", a + b)
```

Output:

```
Element-wise Addition:
0 11
1 22
2 33
dtype: int64
```

Question 2: Write a program to subtract two Series element-wise.

Input:

```
import pandas as pd
a = pd.Series([10, 20, 30])
b = pd.Series([1, 2, 3])
print(a - b)
```

Output:

```
0 9
1 18
2 27
dtype: int64
```

Question 3: Write a program to multiply every value in a DataFrame by a constant.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [2, 3, 4], "B": [5, 6, 7]})
print(df * 2)
```

Output:

```
A B
0 4 10
1 6 12
2 8 14
```

Question 4: Write a program to divide two Series element-wise.

Input:

```
import pandas as pd
a = pd.Series([10, 20, 30])
b = pd.Series([2, 4, 5])
print(a / b)
```

Output:

```
0 5.0
1 5.0
2 6.0
dtype: float64
```

6.2 Column-to-Column Operations

New columns can be created by directly combining two existing columns of a DataFrame with an arithmetic operator.

Question 1: Write a program to compute a total column from price and quantity columns.

Input:

```
import pandas as pd
df = pd.DataFrame({"Price": [100, 200], "Qty": [2, 3]})
df["Total"] = df["Price"] * df["Qty"]
print(df)
```

Output:

```
Price Qty Total
0 100 2 200
1 200 3 600
```

Question 2: Write a program to compute the difference between two columns.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [10, 20], "B": [3, 4]})
df["Diff"] = df["A"] - df["B"]
print(df)
```

Output:

```
A B Diff
0 10 3 7
1 20 4 16
```

Question 3: Write a program to compute a percentage column from marks and totals.

Input:

```
import pandas as pd
df = pd.DataFrame({"Marks": [80, 90], "Total": [100, 100]})
df["Pct"] = df["Marks"] / df["Total"] * 100
print(df)
```

Output:

```
Marks Total Pct
0 80 100 80.0
1 90 100 90.0
```

Question 4: Write a program to compute the square of a column.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
df["Squared"] = df["A"] ** 2
print(df)
```

Output:

```
A Squared
0 1 1
1 2 4
2 3 9
```

7. Handling Missing Data

7.1 isna() & notna()

isna() marks every missing (NaN) value in a DataFrame as True, while notna() marks every present value as True.

Question 1: Write a program to identify missing values in a DataFrame using isna().

Input:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"A": [1, np.nan, 3]})
print(df.isna())
```

Output:

```
A
0 False
1 True
2 False
```

Question 2: Write a program to identify non-missing values in a DataFrame using notna().

Input:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"A": [1, np.nan, 3]})
print(df.notna())
```

Output:

```
A
0 True
1 False
2 True
```

Question 3: Write a program to count the number of missing values in a Series.

Input:

```
import pandas as pd
import numpy as np
s_obj = pd.Series([np.nan, 2, np.nan])
print(s_obj.isna().sum())
```

Output:

```
2
```

Question 4: Write a program to check which columns of a DataFrame contain any missing values.

Input:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"A": [1, np.nan], "B": [np.nan, 4]})
print(df.isna().any())
```

Output:

```
A True
B True
dtype: bool
```

7.2 fillna() & dropna()

fillna() replaces every missing value with a chosen value, while dropna() removes rows or columns that contain missing values entirely.

Question 1: Write a program to fill missing values in a DataFrame with zero.

Input:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"A": [1, np.nan, 3]})
print(df.fillna(0))
```

Output:

```
A
0 1.0
1 0.0
2 3.0
```

Question 2: Write a program to drop rows containing missing values.

Input:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"A": [1, np.nan, 3]})
print(df.dropna())
```

Output:

```
A
0 1.0
2 3.0
```

Question 3: Write a program to fill missing values in a Series with its mean.

Input:

```
import pandas as pd
import numpy as np
s_obj = pd.Series([1, np.nan, 3, np.nan])
print(s_obj.fillna(s_obj.mean()))
```

Output:

```
0 1.0
1 2.0
2 3.0
3 2.0
dtype: float64
```

Question 4: Write a program to drop columns containing missing values.

Input:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"A": [1, np.nan], "B": [3, 4]})
print(df.dropna(axis=1))
```

Output:

```
B
0 3
1 4
```

8. Statistical Functions

8.1 sum(), mean(), median()

sum() adds up all values in a column, mean() computes the average, and median() finds the middle value once sorted.

Question 1: Write a program to compute the sum and mean of a column.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [10, 50, 20, 40]})
print("Sum:", df["A"].sum())
print("Mean:", df["A"].mean())
```

Output:

```
Sum: 120
Mean: 30.0
```

Question 2: Write a program to compute the median of a column.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3, 4, 5]})
print(df["A"].median())
```

Output:

3.0

Question 3: Write a program to compute the sum of each column in a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(df.sum(axis=0))
```

Output:

```
A 3
B 7
dtype: int64
```

Question 4: Write a program to compute the mean of each row in a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(df.mean(axis=1))
```

Output:

```
0 2.0
1 3.0
dtype: float64
```

8.2 min(), max(), idxmin(), idxmax()

min() and max() report the smallest and largest values in a column, while idxmin() and idxmax() report the index labels where those values occur.

Question 1: Write a program to find the minimum value and the index of the maximum value in a column.

Input:

```
import pandas as pd
df = pd.DataFrame({"Score": [45, 89, 23, 71]})
print("Minimum:", df["Score"].min())
print("Index of Max:", df["Score"].idxmax())
```

Output:

```
Minimum: 23  
Index of Max: 1
```

Question 2: Write a program to find the maximum value in a column.

Input:

```
import pandas as pd  
df = pd.DataFrame({"Score": [45, 89, 23, 71]})  
print(df["Score"].max())
```

Output:

```
89
```

Question 3: Write a program to find the index of the minimum value in a column.

Input:

```
import pandas as pd  
df = pd.DataFrame({"Score": [45, 89, 23, 71]})  
print(df["Score"].idxmin())
```

Output:

```
2
```

Question 4: Write a program to find the maximum value of each column in a DataFrame.

Input:

```
import pandas as pd  
df = pd.DataFrame({"A": [3, 7], "B": [9, 1]})  
print(df.max(axis=0))
```

Output:

```
A 7  
B 9  
dtype: int64
```

8.3 value_counts()

`value_counts()` counts how many times each unique value appears in a Series, returning the results sorted from most to least frequent.

Question 1: Write a program to count the occurrences of each value in a Series.

Input:

```
import pandas as pd
s_obj = pd.Series(["a", "b", "a", "c", "b", "a"])
print(s_obj.value_counts())
```

Output:

```
a 3
b 2
c 1
Name: count, dtype: int64
```

Question 2: Write a program to count the occurrences of each city in a DataFrame column.

Input:

```
import pandas as pd
df = pd.DataFrame({"City": ["Delhi", "Pune", "Delhi", "Goa"]})
print(df["City"].value_counts())
```

Output:

```
City
Delhi 2
Pune 1
Goa 1
Name: count, dtype: int64
```

Question 3: Write a program to compute the normalized frequency of values in a Series.

Input:

```
import pandas as pd
s_obj = pd.Series([1, 1, 2, 3, 3, 3])
print(s_obj.value_counts(normalize=True))
```

Output:

```
3 0.500000
1 0.333333
2 0.166667
Name: proportion, dtype: float64
```

Question 4: Write a program to find the most frequent value in a Series.

Input:

```
import pandas as pd
s_obj = pd.Series(["x", "y", "x"])
print(s_obj.value_counts().index[0])
```

Output:

```
x
```

9. Sorting

9.1 sort_values()

sort_values() returns a new DataFrame or Series with rows arranged in ascending order based on a chosen column, by default.

Question 1: Write a program to sort a DataFrame by age in ascending order.

Input:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Ben", "Ann", "Cid"], "Age": [31, 23, 27]})
print(df.sort_values("Age"))
```

Output:

```
Name Age
1 Ann 23
2 Cid 27
0 Ben 31
```

Question 2: Write a program to sort a DataFrame in descending order.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [3, 1, 2]})
print(df.sort_values("A", ascending=False))
```

Output:

```
A
0 3
2 2
1 1
```

Question 3: Write a program to sort a Series by its values.

Input:

```
import pandas as pd
s_obj = pd.Series([5, 1, 4, 2, 3])
print(s_obj.sort_values())
```

Output:

```
1 1
3 2
4 3
2 4
0 5
dtype: int64
```

Question 4: Write a program to sort a DataFrame by two columns.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 1, 2], "B": [30, 10, 20]})
print(df.sort_values(["A", "B"]))
```

Output:

```
A B
1 1 10
0 1 30
2 2 20
```

9.2 sort_index()

sort_index() returns a new DataFrame or Series with rows rearranged according to their index labels rather than their values.

Question 1: Write a program to sort a DataFrame by its index.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]}, index=[3, 1, 2])
print(df.sort_index())
```

Output:

```
A
1 2
2 3
3 1
```

Question 2: Write a program to sort a Series by its index labels.

Input:

```
import pandas as pd
s_obj = pd.Series([10, 20, 30], index=["c", "a", "b"])
print(s_obj.sort_index())
```

Output:

```
a 20
b 30
c 10
dtype: int64
```

Question 3: Write a program to sort a DataFrame by index in descending order.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]}, index=[3, 1, 2])
print(df.sort_index(ascending=False))
```

Output:

```
A
3 1
```

```
2 3
1 2
```

Question 4: Write a program to sort a DataFrame's columns alphabetically in descending order.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2]}, columns=["A"])
df["B"] = [3, 4]
print(df.sort_index(axis=1, ascending=False))
```

Output:

```
B A
0 3 1
1 4 2
```

10. Grouping

10.1 groupby()

groupby() splits a DataFrame into groups based on the values of one or more columns, so an aggregate function can be applied to each group separately.

Question 1: Write a program to compute total sales for each city using groupby().

Input:

```
import pandas as pd
df = pd.DataFrame({"City": ["Delhi", "Pune", "Delhi"], "Sales": [100, 150, 200]})
print(df.groupby("City")["Sales"].sum())
```

Output:

```
City
Delhi 300
Pune 150
Name: Sales, dtype: int64
```

Question 2: Write a program to compute the average points for each team using `groupby()`.

Input:

```
import pandas as pd
df = pd.DataFrame({"Team": ["A", "B", "A", "B"], "Points": [10, 20, 30, 40]})
print(df.groupby("Team")["Points"].mean())
```

Output:

```
Team
A 20.0
B 30.0
Name: Points, dtype: float64
```

Question 3: Write a program to count the number of rows in each group.

Input:

```
import pandas as pd
df = pd.DataFrame({"Cat": ["x", "y", "x"], "Val": [1, 2, 3]})
print(df.groupby("Cat").size())
```

Output:

```
Cat
x 2
y 1
dtype: int64
```

Question 4: Write a program to compute multiple aggregate statistics for each group.

Input:

```
import pandas as pd
df = pd.DataFrame({"Cat": ["x", "y", "x"], "Val": [1, 2, 3]})
print(df.groupby("Cat")["Val"].agg(["sum", "mean"]))
```

Output:

```
sum mean
Cat
x 4 2.0
y 2 2.0
```

11. Merging & Joining

11.1 pd.merge()

pd.merge() combines two DataFrames into one based on matching values in a shared key column, similar to a SQL join.

Question 1: Write a program to merge two DataFrames on a common id column.

Input:

```
import pandas as pd
left = pd.DataFrame({"id": [1, 2], "name": ["Ann", "Ben"]})
right = pd.DataFrame({"id": [1, 2], "score": [90, 85]})
print(pd.merge(left, right, on="id"))
```

Output:

```
id name score
0 1 Ann 90
1 2 Ben 85
```

Question 2: Write a program to perform an inner join between two DataFrames.

Input:

```
import pandas as pd
left = pd.DataFrame({"id": [1, 2, 3], "val": ["a", "b", "c"]})
right = pd.DataFrame({"id": [2, 3, 4], "extra": ["x", "y", "z"]})
print(pd.merge(left, right, on="id", how="inner"))
```

Output:

```
id val extra
0 2 b x
1 3 c y
```

Question 3: Write a program to perform a left join between two DataFrames.

Input:

```
import pandas as pd
left = pd.DataFrame({"id": [1, 2], "val": ["a", "b"]})
right = pd.DataFrame({"id": [2, 3], "extra": ["x", "y"]})
```

```
print(pd.merge(left, right, on="id", how="left"))
```

Output:

```
id val extra  
0 1 a NaN  
1 2 b x
```

Question 4: Write a program to perform an outer join between two DataFrames.

Input:

```
import pandas as pd  
left = pd.DataFrame({"id": [1, 2], "val": ["a", "b"]})  
right = pd.DataFrame({"id": [2, 3], "extra": ["x", "y"]})  
print(pd.merge(left, right, on="id", how="outer"))
```

Output:

```
id val extra  
0 1 a NaN  
1 2 b x  
2 3 NaN y
```

11.2 pd.concat()

pd.concat() stacks multiple DataFrames together either vertically (adding rows) or horizontally (adding columns).

Question 1: Write a program to concatenate two DataFrames vertically.

Input:

```
import pandas as pd  
df1 = pd.DataFrame({"A": [1, 2]})  
df2 = pd.DataFrame({"A": [3, 4]})  
print(pd.concat([df1, df2], ignore_index=True))
```

Output:

```
A  
0 1  
1 2  
2 3
```

3 4

Question 2: Write a program to concatenate two DataFrames horizontally.

Input:

```
import pandas as pd
df1 = pd.DataFrame({"A": [1, 2]})
df2 = pd.DataFrame({"B": [3, 4]})
print(pd.concat([df1, df2], axis=1))
```

Output:

```
A B
0 1 3
1 2 4
```

Question 3: Write a program to concatenate three DataFrames into one.

Input:

```
import pandas as pd
df1 = pd.DataFrame({"A": [1]})
df2 = pd.DataFrame({"A": [2]})
df3 = pd.DataFrame({"A": [3]})
print(pd.concat([df1, df2, df3], ignore_index=True))
```

Output:

```
A
0 1
1 2
2 3
```

Question 4: Write a program to concatenate two DataFrames with custom row indexes.

Input:

```
import pandas as pd
df1 = pd.DataFrame({"A": [1, 2]}, index=["r1", "r2"])
df2 = pd.DataFrame({"A": [3, 4]}, index=["r3", "r4"])
print(pd.concat([df1, df2]))
```

Output:

```
A
r1 1
r2 2
r3 3
r4 4
```

12. Applying Functions

12.1 apply() & map()

`apply()` runs a custom function across every row, column, or value of a DataFrame, while `map()` runs a function or lookup across every value of a Series.

Question 1: Write a program to multiply every value in a column using `apply()`.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
print(df["A"].apply(lambda x: x * 10))
```

Output:

```
0 10
1 20
2 30
Name: A, dtype: int64
```

Question 2: Write a program to map numeric values in a Series to their word equivalents.

Input:

```
import pandas as pd
s_obj = pd.Series([1, 2, 3])
print(s_obj.map({1: "one", 2: "two", 3: "three"}))
```

Output:

```
0 one
1 two
2 three
```

```
dtype: str
```

Question 3: Write a program to compute the sum of each column using `apply()`.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(df.apply(lambda col: col.sum(), axis=0))
```

Output:

```
A 3
B 7
dtype: int64
```

Question 4: Write a program to compute the sum of two columns for each row using `apply()`.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
print(df.apply(lambda row: row["A"] + row["B"], axis=1))
```

Output:

```
0 4
1 6
dtype: int64
```

13. Pivot Tables

13.1 `pivot_table()`

`pivot_table()` reshapes a DataFrame by grouping rows and columns around two chosen fields while aggregating a third value field.

Question 1: Write a program to create a pivot table of sales by city and month.

Input:

```
import pandas as pd
df = pd.DataFrame({
    "City": ["Delhi", "Delhi", "Pune", "Pune"],
    "Month": ["Jan", "Feb", "Jan", "Feb"],
    "Sales": [100, 150, 80, 120]
})
print(df.pivot_table(index="City", columns="Month", values="Sales"))
```

Output:

```
Month Feb Jan
City
Delhi 150.0 100.0
Pune 120.0 80.0
```

Question 2: Write a program to create a pivot table summing values by team.

Input:

```
import pandas as pd
df = pd.DataFrame({"Team": ["A", "A", "B"], "Val": [10, 20, 30]})
print(df.pivot_table(index="Team", values="Val", aggfunc="sum"))
```

Output:

```
Val
Team
A 30
B 30
```

Question 3: Write a program to create a pivot table averaging values by category.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": ["x", "x", "y"], "B": [1, 2, 3]})
print(df.pivot_table(index="A", values="B", aggfunc="mean"))
```

Output:

```
B
A
x 1.5
y 3.0
```

Question 4: Write a program to create a pivot table with missing combinations filled with zero.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": ["x", "y"], "B": ["p", "q"], "V": [1, 2]})
print(df.pivot_table(index="A", columns="B", values="V", fill_value=0))
```

Output:

```
B p q
A
x 1.0 0.0
y 0.0 2.0
```

14. String Operations

14.1 The .str accessor

The .str accessor exposes vectorized, element-wise string methods that run across every value in a Series without an explicit loop.

Question 1: Write a program to convert a Series of strings to uppercase.

Input:

```
import pandas as pd
s_obj = pd.Series(["ann", "ben", "cid"])
print(s_obj.str.upper())
```

Output:

```
0 ANN
1 BEN
2 CID
dtype: str
```

Question 2: Write a program to strip whitespace from a Series of strings.

Input:

```
import pandas as pd
s_obj = pd.Series([" hello ", " world "])
```

```
print(s_obj.str.strip())
```

Output:

```
0 hello
1 world
dtype: str
```

Question 3: Write a program to check which strings contain a given substring.

Input:

```
import pandas as pd
s_obj = pd.Series(["cat", "caterpillar", "dog"])
print(s_obj.str.contains("cat"))
```

Output:

```
0 True
1 True
2 False
dtype: bool
```

Question 4: Write a program to split a Series of strings on a delimiter.

Input:

```
import pandas as pd
s_obj = pd.Series(["one-two", "three-four"])
print(s_obj.str.split("-"))
```

Output:

```
0 [one, two]
1 [three, four]
dtype: object
```

15. Date & Time Handling

15.1 dt accessor

The .dt accessor exposes date and time properties, such as year, month, and weekday, for every Timestamp stored in a datetime Series.

Question 1: Write a program to extract the year from a datetime Series.

Input:

```
import pandas as pd
s_obj = pd.Series(pd.to_datetime(["2024-01-15", "2024-06-20"]))
print(s_obj.dt.year)
```

Output:

```
0 2024
1 2024
dtype: int32
```

Question 2: Write a program to extract the month from a datetime Series.

Input:

```
import pandas as pd
s_obj = pd.Series(pd.to_datetime(["2024-01-15", "2024-06-20"]))
print(s_obj.dt.month)
```

Output:

```
0 1
1 6
dtype: int32
```

Question 3: Write a program to extract the day name from a datetime Series.

Input:

```
import pandas as pd
s_obj = pd.Series(pd.to_datetime(["2024-01-15", "2024-06-20"]))
print(s_obj.dt.day_name())
```

Output:

```
0 Monday
1 Thursday
dtype: str
```

Question 4: Write a program to extract the day of the year from a datetime Series.

Input:

```
import pandas as pd
s_obj = pd.Series(pd.to_datetime(["2024-01-01", "2024-12-31"]))
print(s_obj.dt.dayofyear)
```

Output:

```
0 1
1 366
dtype: int32
```

16. Duplicates & Unique Values

16.1 unique(), nunique(), duplicated(), drop_duplicates()

unique() lists the distinct values in a Series, nunique() counts them, duplicated() flags repeated rows, and drop_duplicates() removes them.

Question 1: Write a program to list the unique values in a Series.

Input:

```
import pandas as pd
s_obj = pd.Series(["a", "b", "a", "c", "b"])
print(s_obj.unique())
```

Output:

```
<StringArray>
['a', 'b', 'c']
Length: 3, dtype: str
```

Question 2: Write a program to count the unique values in a Series.

Input:

```
import pandas as pd
s_obj = pd.Series(["a", "b", "a", "c", "b"])
print(s_obj.nunique())
```

Output:

```
3
```

Question 3: Write a program to identify duplicate rows in a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 1, 2], "B": [3, 3, 4]})
print(df.duplicated())
```

Output:

```
0 False
1 True
2 False
dtype: bool
```

Question 4: Write a program to remove duplicate rows from a DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 1, 2], "B": [3, 3, 4]})
print(df.drop_duplicates())
```

Output:

```
A B
0 1 3
2 2 4
```

17. File Handling

17.1 to_csv() & read_csv()

to_csv() writes a DataFrame out to a plain text CSV file on disk, and read_csv() reads that file back into memory as a DataFrame.

Question 1: Write a program to save a DataFrame to a CSV file and print its shape after reloading.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [5, 10], "B": [15, 20]})
df.to_csv("data_log.csv", index=False)
```

```
loaded = pd.read_csv("data_log.csv")
print("Loaded Shape:", loaded.shape)
```

Output:

```
Loaded Shape: (2, 2)
```

Question 2: Write a program to save a numeric Series to CSV and reload it.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3, 4]})
df.to_csv("vector_log.csv", index=False)
print(pd.read_csv("vector_log.csv"))
```

Output:

```
A
0 1
1 2
2 3
3 4
```

Question 3: Write a program to save a float DataFrame to CSV with two decimal places.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1.5, 2.5], "B": [3.5, 4.5]})
df.to_csv("floats_log.csv", index=False, float_format="%.2f")
print(pd.read_csv("floats_log.csv"))
```

Output:

```
A B
0 1.5 3.5
1 2.5 4.5
```

Question 4: Write a program to save and reload a two-column numeric DataFrame.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [0, 1], "B": [2, 3]})
df.to_csv("grid_log.csv", index=False)
print(pd.read_csv("grid_log.csv"))
```

Output:

```
A B
0 0 2
1 1 3
```

17.2 to_excel() & read_excel()

to_excel() writes a DataFrame out to an Excel workbook, and read_excel() reads a sheet from an Excel workbook back into a DataFrame.

Question 1: Write a program to save a DataFrame to an Excel file and read it back.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
df.to_excel("sample.xlsx", index=False)
loaded = pd.read_excel("sample.xlsx")
print(loaded)
```

Output:

```
A B
0 1 3
1 2 4
```

Question 2: Write a program to save a DataFrame to a named Excel sheet and read it back.

Input:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Ann", "Ben"], "Age": [23, 31]})
df.to_excel("people.xlsx", index=False, sheet_name="People")
loaded = pd.read_excel("people.xlsx", sheet_name="People")
print(loaded)
```

Output:

```
Name Age
0 Ann 23
1 Ben 31
```

Question 3: Write a program to save a float column to Excel and check its data type after reloading.

Input:

```
import pandas as pd
df = pd.DataFrame({"Val": [1.1, 2.2, 3.3]})
df.to_excel("vals.xlsx", index=False)
loaded = pd.read_excel("vals.xlsx")
print(loaded.dtypes)
```

Output:

```
Val float64
dtype: object
```

Question 4: Write a program to save a DataFrame to Excel without the index and print its shape after reloading.

Input:

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3]})
df.to_excel("noidx.xlsx", index=False)
loaded = pd.read_excel("noidx.xlsx")
print(loaded.shape)
```

Output:

```
(3, 1)
```